

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1
“Основные конструкции языка Python ”

Выполнил:
Студент группы ИУ5-34Б
Гудис А.В.
Преподаватель:
Нардид А.Н.

Москва 2025

Задание

Разработать программу для решения [биквадратного уравнения](#).

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов А, В, С, вычисляет дискриминант и **ДЕЙСТВИТЕЛЬНЫЕ** корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты А, В, С могут быть заданы в виде параметров командной строки ([вариант задания параметров приведен в конце файла с примером кода](#)). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. [Описание работы с параметрами командной строки](#).
4. Если коэффициент А, В, С введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования (кроме C++).

Код(lab1)

```
import sys
import math
from typing import NamedTuple

# Алгебраический тип
class SquareRootResult:
    NoRoots = NamedTuple("NoRoots", [])
    OneRoot = NamedTuple("OneRoot", [("root", float)])
    TwoRoots = NamedTuple("TwoRoots", [("root1", float), ("root2", float)])
    ThreeRoots = NamedTuple("ThreeRoots", [("root1", float), ("root2", float), ("root3", float)])
    FourRoots = NamedTuple("FourRoots", [("root1", float), ("root2", float), ("root3", float), ("root4", float)])

def get_coef(index, prompt):
    """
    Читаем коэффициент из командной строки или вводим с клавиатуры

    Args:
        index (int): Номер параметра в командной строке
    """
```

prompt (str): Приглашение для ввода коэффициента

Returns:

float: Коэффициент квадратного уравнения
'''

try:

Пробуем прочитать коэффициент из командной строки

coef_str = sys.argv[index]

except:

Вводим с клавиатуры

print(prompt)

coef_str = input()

Переводим строку в действительное число

coef = float(coef_str)

return coef

def get_roots_t(a,b,c,t):

if t==0:

roots = {0}

return roots

elif t<0:

return {}

roots = {t**0.5, t**0.5*-1}

return roots

def get_roots(a, b, c):

'''

Вычисление корней квадратного уравнения

Args:

a (float): коэффициент A

b (float): коэффициент B

c (float): коэффициент C

Returns:

Список корней в виде типа SquareRootResult

'''

D = b*b - 4*a*c

if D == 0.0:

t = -b / (2.0*a)

roots = get_roots_t(a,b,c,t)

elif D > 0.0:

sqD = math.sqrt(D)

t1 = (-b+sqD)/(2*a)

t2 = (-b-sqD)/(2*a)

roots1 = get_roots_t(a,b,c,t1)

roots2 = get_roots_t(a,b,c,t2)

roots = roots1.union(roots2)

else:

return SquareRootResult.NoRoots()

```

roots_list = list(roots)
if len(roots_list)==4:
    return SquareRootResult.FourRoots(roots_list[0], roots_list[1], roots_list[2], roots_list[3])
if len(roots_list)==3:
    return SquareRootResult.ThreeRoots(roots_list[0], roots_list[1], roots_list[2])
elif len(roots_list)==2:
    return SquareRootResult.TwoRoots(roots_list[0], roots_list[1])
else:
    return SquareRootResult.OneRoot(roots_list[0])

def print_roots(roots_tuple):
    """
    Печать корней квадратного уравнения

    Args:
        Список корней в виде типа SquareRootResult
    """
    match roots_tuple:
        case SquareRootResult.FourRoots(root1, root2, root3, root4):
            print(f'Четыре корня: {root1}, {root2}, {root3} и {root4}')
        case SquareRootResult.ThreeRoots(root1, root2, root3):
            print(f'Три корня: {root1}, {root2} и {root3}')
        case SquareRootResult.TwoRoots(root1, root2):
            print(f'Два корня: {root1} и {root2}')
        case SquareRootResult.OneRoot(root):
            print(f'Один корень: {root}')
        case SquareRootResult.NoRoots():
            print('Нет корней')

def main():
    """
    Основная функция
    """
    a = get_coef(1, 'Введите коэффициент А:')
    b = get_coef(2, 'Введите коэффициент В:')
    c = get_coef(3, 'Введите коэффициент С:')
    # Вычисление корней
    roots = get_roots(a,b,c)
    # Вывод корней
    print_roots(roots)

if __name__ == "__main__":
    main()

```

Работа приложения

```
Введите коэффициент A:  
12  
Введите коэффициент B:  
13  
Введите коэффициент C:  
14  
Нет корней  
  
Process finished with exit code 0
```

```
(.venv) PS C:\Users\ADM\PycharmProjects\Zherebenkov-PkiyapLabs\lab1> python main.py 1 2 3  
Нет корней
```

```
Введите коэффициент A:  
3  
Введите коэффициент B:  
8  
Введите коэффициент C:  
-3  
Два корня: 0.5773502691896257 и -0.5773502691896257
```